

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de

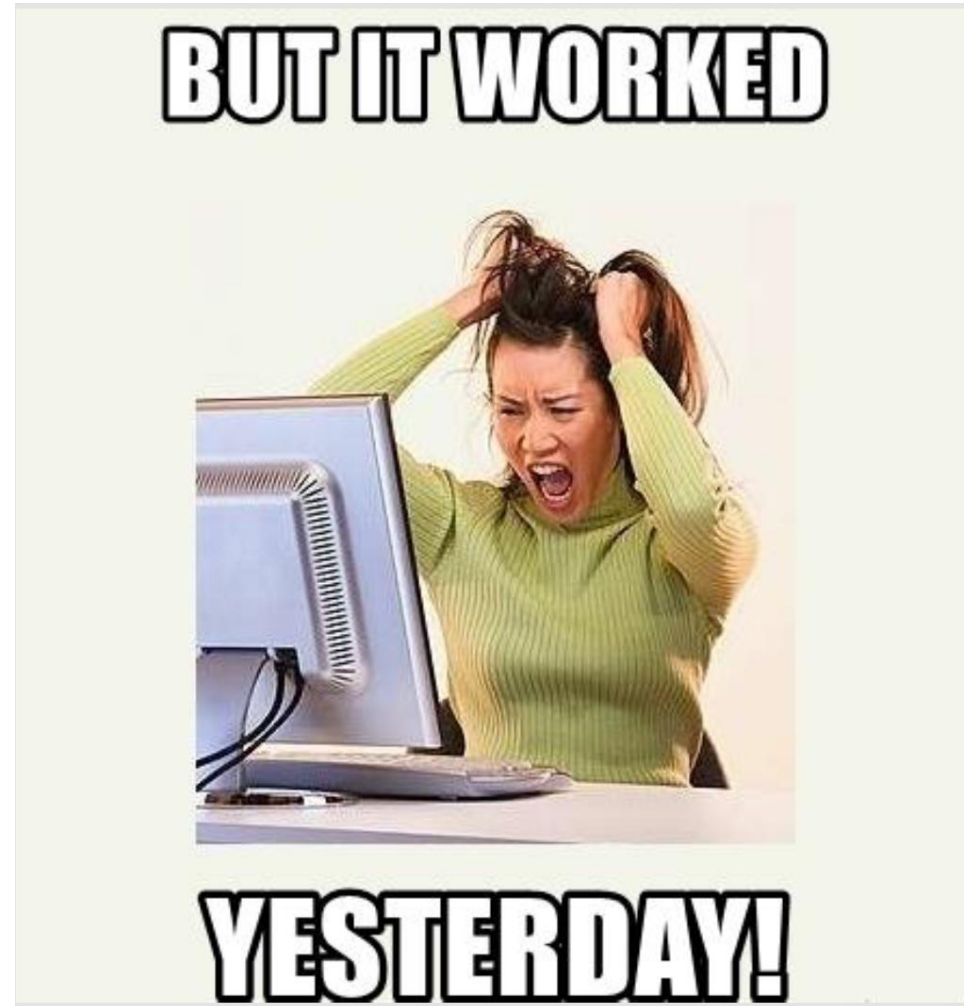


Kapitel 2: Objektorientierung

1. Grundlagen
2. Header Files
3. Vererbung
4. Polymorphismus

Exkurs: Git

Versionsverwaltung mit Git



Git

- Open-source Versionskontroll-Software, um Änderungen zu verfolgen
- Bietet Möglichkeit ...
 - Zu jedem gespeicherten alten Stand zurückzukehren
 - Den Unterschied zwischen zwei Ständen zu visualisieren
 - Gleichzeitiges Arbeiten am gleichen Code mit Handeln von Konflikten etc.
- Hier:
 - Erste Erfahrungen mit Git
 - Regelmäßige Sicherheitskopien
 - Arbeiten zu zweit an Übungsprojekt
 - Verteilung von Beispielaufgaben und Vorlesungsmaterial

Download Git for Windows

- **Git**

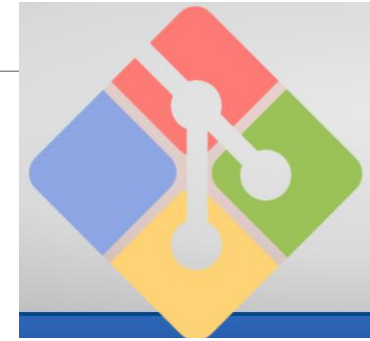
- Git bash for Windows Download [hier](#)
- Installiert Git und eine Konsole (Git Bash)

- **Github**

- Account kann [hier](#) erstellt werden

- **Alternative Anbieter zu Github**

- Gitlab.com (für kleine Teams gratis)
- Bitbucket.org (für kleine Teams gratis)



Git – Demo: Neues Repository erstellen

- Github → New repository
- Name, Beschreibung, privat/public, add .gitignore file for C++

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner * mdrue04 / Repository name * cpp_dhbw ⚠

Great repository names are The repository cpp_dhbw already exists on this account. out [curly-umbrella?](#)

Description (optional)

Cpp lecture 2022

☐ Public
Anyone on the internet can see this repository. You choose who can commit.

☒ Private
You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☒ Add a README file
This is where you can write a long description for your project. [Learn more.](#)

☒ Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

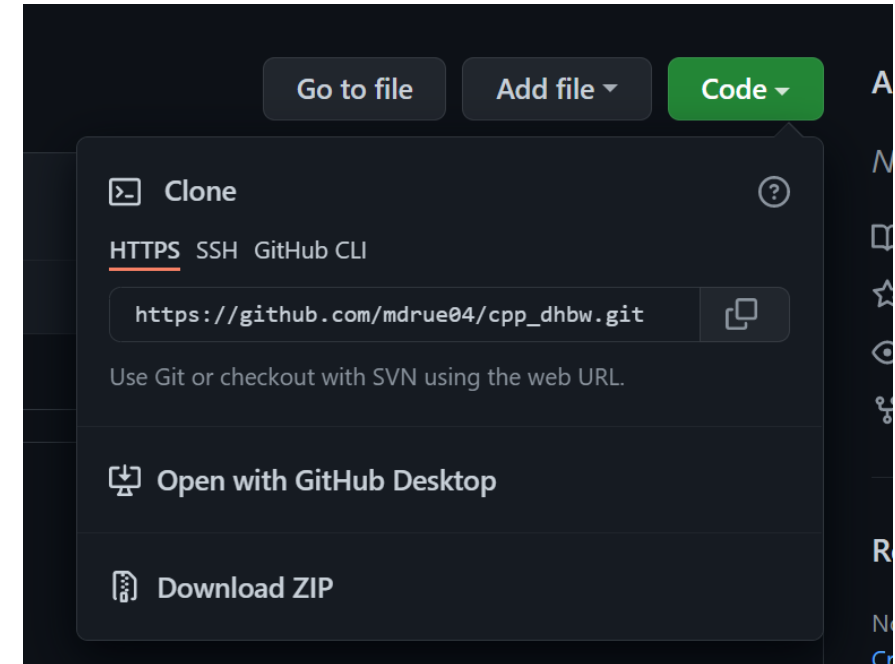
.gitignore template: C++

☐ Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

Create repository

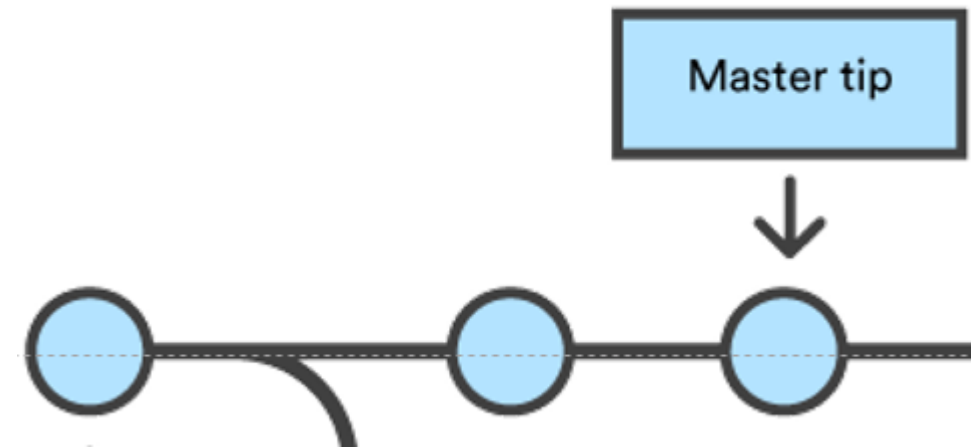
Github – Neues Git Repository

- Gehen Sie auf Code
 - Dann unter Clone, den Text unter HTTPS in die Zwischenablage kopieren
- In einer Git Console (z.B. git Bash)
 - change directory (cd) um zum richtigen Ordner zu kommen
- Dann in der Konsole:
`$git clone <https_text>`
- Dies legt bei Ihnen eine lokale Kopie des Git-Repos an



Git - Commits

- Commit: 
 - im Git Repository „registrierte“ Sammlung an Änderungen
 - + Autor
 - + Zeitstempel
 - + Commit Message
 - Entspricht einer „Momentaufnahme“ der Projektcodes



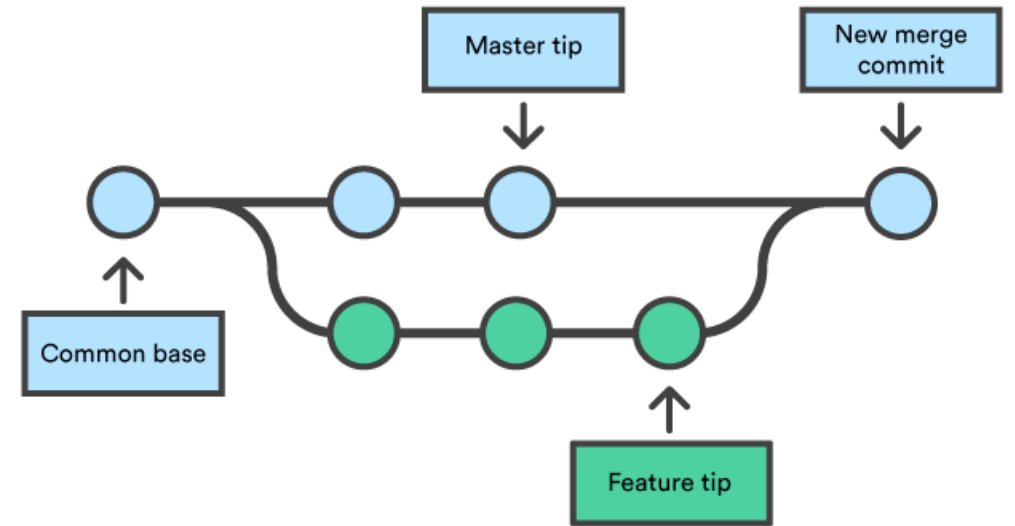
Git – Demo: Commits

Inhalt der Demo

- Show git repository in github and in git bash
- Add new file, make changes
- Commit and push changes in bash and in VS Code
- Show changes online

Git - Branches

- Um gleichzeitig an unterschiedlichen Ständen arbeiten zu können gibt es Branches
- Der Master-Branch ist der Haupt-Branch
- Davon können neue Branches abzweigen (z.B. für ein Feature, d.h. eine in sich abgeschlossene Erweiterung eines Programms)
- Darauf können erneut Commits erstellt werden
- Diese Commits sind nur dort verfügbar
- Ist das Feature fertig implementiert, kann man mit einem „merge“ (oft über einen Pull-Request) die Commits auf den Master „zurückführen“
- Um Stabilität zu gewährleisten wird der Master Branch gewöhnlich geschützt (Code Reviews, Pull-Request Builds, ...)



Achtung: Es gibt immer eine Server Version (origin/) eines Branches und eine lokale Version. Diese sind z.B. so lange unterschiedlich, bis lokale Commits gepusht werden oder Server Commits gepullt werden (nächste Folie)

Git – wichtige Befehle

- Git ist entweder über Konsole oder über ein GUI steuerbar, hier die Konsolen Befehle
- `git status` : Zeigt den Status des lokalen Repositories an
- `git add <fileName>/--all` : fügt <fileName>/alles der Staging Area hinzu
- `git commit -m „Commit Message“` : Erstellt einen Commit (erst noch lokal)
- `git push` : lädt die lokalen Commits des aktuellen Branchs hoch
- `git checkout <branchName>` : wechseln auf branchName
- `git fetch` : aktualisiert die lokale Kopie von origin/<current_branch>
- `git pull` : hole die neuesten Commits des aktuellen Branchs vom Server auf die lokale Kopie des Branches

Git – Demo: Branches

- Mehr Details auch z.B. in dieser [Anleitung](#)

Inhalt der Demo:

- Create new branch in git bash: `git checkout -b <branch_name>`
- Show new branch in github: `git checkout <branch_name>`
- Make, commit and push changes
- Open PR and merge branch
- Git Bash: checkout, status, fetch, pull
- Checkout older version of branch and remove new changes: `git reset --hard <commit_id>`

Git - Wann wird committed?

- Ein Commit ist die kleinste Einheit von Änderungen die verglichen und z.B. auch wieder zurück genommen werden können. Diese sollte nicht zu klein auch auf keinen Fall zu groß sein.
- Wenn eine logisch zusammenhängende Programmänderung oder ein zusammenhängender Unterpunkt davon abgeschlossen ist (diese kann aus mehreren File-Änderungen bestehen)
- Wenn ich meinen aktuellen Arbeitsstand speichern möchte (weil ich z.B. den Laptop über das Wochenende zu klappe)
- Die Git Message sollte immer eingetragen werden und am besten den Grund für eine Änderung angeben („new commit“ oder „implemented change“ sagen gar nichts aus)

Git – Was wird committed?

- Vorwiegend **Textdateien** (.cpp, .hpp, README, .gitignore, ...)
- Grafiken und Schaubilder (können in README eingebettet werden)
- Keine großen Daten (lediglich Beispieldaten)
- Keine Executables
- Änderungen können im Binärformat nicht nachvollzogen werden
→ nicht sinnvoll für Versionsverwaltung

Aufgaben: Git

- 1) Legen Sie sich bei einem Anbieter ihrer Wahl einen Git Account und ein Repository für die Vorlesung an (Support nur für Github).
- 2) Checken Sie dieses aus und machen Sie sich mit der Funktionsweise vertraut. Es genügt, wenn Sie danach in der Lage sind:
 - a) Einen Commit mit einer Nachricht anzulegen und zu pushen
 - b) Die Änderungen eines älteren Commits einzusehen
 - c) Zu einem älteren Commit zurückzuspringen könnt (`git checkout <commit_id>`)
 - d) Einen Branch zu erstellen, dort zu commiten und zu pushen und diesen wieder zu mergen
- 3) Committen Sie den Code aus der letzten Aufgabe in Ihrem Repo und machen Sie mindestens eine neue Änderung die auch gepusht wird.